

Here are a few project ideas for a digital image processing lab along with brief explanations:

Image Filtering and Enhancement:

1. Implement various image filtering techniques such as mean filtering, median filtering, and Gaussian filtering. Apply these filters to an image and compare their effects on image quality and noise reduction. Additionally, explore enhancement techniques like histogram equalization and contrast stretching to improve the visual appearance of images.

Image Segmentation:

2. Develop an image segmentation algorithm to partition an image into meaningful regions or objects. Use techniques like thresholding, region growing, or edge detection to separate foreground objects from the background. Evaluate the performance of different segmentation methods on different types of images.

Image Compression:

3. Explore image compression techniques such as JPEG or wavelet-based compression. Implement and compare the performance of different compression algorithms in terms of compression ratio, image quality, and computational complexity. Analyze the trade-offs between compression efficiency and visual fidelity.

Image Restoration:

4. Design an image restoration system to remove noise or artifacts from degraded images. Implement techniques such as inverse filtering, Wiener filtering, or total variation regularization. Evaluate the effectiveness of these methods in restoring images corrupted by various types of noise or degradation.

Image Recognition:

5. Build an image recognition system using techniques like feature extraction and classification. Train a machine learning model, such as a convolutional neural network (CNN), to classify images into different categories or objects. Evaluate the accuracy and performance of the recognition system on benchmark datasets.

Medical Image Analysis:

6. Focus on medical image processing and analysis. Implement algorithms for medical image segmentation, tumor detection, or organ recognition. Explore techniques like region-based methods, active contour models, or machine learning approaches specific to medical imaging. Evaluate the performance of the algorithms using medical image datasets.

Facial Emotion Recognition:

7. Develop a facial emotion recognition system using facial feature extraction and classification techniques. Use techniques like facial landmark detection, expression analysis, and machine learning algorithms to classify emotions from facial images. Evaluate the system's accuracy and performance on emotion datasets.

Image Steganography:

8. Implement image steganography techniques to hide secret information within an image. Explore methods like LSB (Least Significant Bit) insertion, spread spectrum, or transform domain-based techniques. Evaluate the imperceptibility of the hidden information and the robustness against attacks.

Problem 1:

```
import cv2

import numpy as np


# Load the image

image = cv2.imread('input_image.jpg', 0) # Read image in grayscale


# Mean Filtering

mean_filtered = cv2.blur(image, (5, 5)) # Applying a mean filter with kernel size 5x5


# Median Filtering

median_filtered = cv2.medianBlur(image, 5) # Applying median filter with kernel size 5x5


# Gaussian Filtering

gaussian_filtered = cv2.GaussianBlur(image, (5, 5), 0) # Applying Gaussian filter with kernel size 5x5


# Histogram Equalization

equalized = cv2.equalizeHist(image) # Applying histogram equalization


# Contrast Stretching
```

```
minimum = np.min(image)

maximum = np.max(image)

stretched = cv2.normalize(image, None, 0, 255, cv2.NORM_MINMAX)


# Display the results

cv2.imshow('Original Image', image)

cv2.imshow('Mean Filtered Image', mean_filtered)

cv2.imshow('Median Filtered Image', median_filtered)

cv2.imshow('Gaussian Filtered Image', gaussian_filtered)

cv2.imshow('Histogram Equalized Image', equalized)

cv2.imshow('Contrast Stretched Image', stretched)


cv2.waitKey(0)

cv2.destroyAllWindows()
```

Make sure to replace `'input_image.jpg'` with the actual filename and path of the image you want to process. This code reads the image in grayscale and then applies mean filtering, median filtering, Gaussian filtering, histogram equalization, and contrast stretching techniques. The resulting images are displayed using `cv2.imshow()`.

You'll need to have OpenCV and NumPy installed in your Python environment for this code to work. You can install them using `pip install opencv-python` and `pip install numpy`.

Problem 2:

```
import cv2

import numpy as np


# Load the image

image = cv2.imread('input_image.jpg')

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)


# Thresholding

_, thresholded = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)


# Region Growing

def region_growing(image, seed):

    height, width = image.shape[:2]

    visited = np.zeros((height, width), dtype=np.uint8)

    region = np.zeros((height, width), dtype=np.uint8)

    stack = []

    stack.append(seed)

    while len(stack) > 0:
```

```
x, y = stack.pop()

if x < 0 or y < 0 or x >= height or y >= width:

    continue

if visited[x, y] == 1:

    continue

visited[x, y] = 1

if image[x, y] == 255: # Add condition based on the object you want to segment

    region[x, y] = 255

    stack.append((x + 1, y))

    stack.append((x - 1, y))

    stack.append((x, y + 1))

    stack.append((x, y - 1))

return region

seed_point = (100, 100) # Seed point for region growing
region_grown = region_growing(thresholded, seed_point)

# Display the results
```

```
cv2.imshow('Original Image', image)

cv2.imshow('Thresholded Image', thresholded)

cv2.imshow('Region Grown Image', region_grown)


cv2.waitKey(0)

cv2.destroyAllWindows()
```

Make sure to replace `'input_image.jpg'` with the actual filename and path of the image you want to segment. This code reads the image, converts it to grayscale, performs thresholding to separate the foreground objects from the background, and then applies region growing around a seed point to grow the region of interest.

You can adjust the threshold value in the `cv2.threshold()` function based on the intensity range you want to use for segmentation. Additionally, you can modify the condition inside the `region_growing()` function to select the desired object based on its intensity or color.

This code provides a basic implementation of image segmentation using thresholding and region growing. You can further enhance and customize the segmentation algorithm based on your specific requirements and the characteristics of the images you are working with.

Problem 3

```
from PIL import Image

import numpy as np


# Load the image

image = Image.open('input_image.jpg')


# Convert image to YCbCr color space

image_ycbcr = image.convert('YCbCr')


# Split image into Y, Cb, and Cr channels

y, cb, cr = image_ycbcr.split()


# Perform block-based DCT on each channel

block_size = 8

y_blocks = np.asarray(y).reshape(-1, block_size,
block_size).astype(np.float32)

cb_blocks = np.asarray(cb).reshape(-1, block_size,
block_size).astype(np.float32)
```



```
cr_blocks = np.asarray(cr).reshape(-1, block_size,  
block_size).astype(np.float32)
```

```
# Quantization
```

```
# Apply quantization tables specific to each channel
```

```
# Entropy coding (Huffman encoding)
```

```
# Calculate compression ratio
```

```
original_size = image.size[0] * image.size[1] * 3 # Original image size in  
bytes
```

```
compressed_size = len(compressed_y) + len(compressed_cb) +  
len(compressed_cr) # Compressed image size in bytes
```

```
compression_ratio = original_size / compressed_size
```

```
# Reconstruct image
```

```
# Apply inverse quantization
```

```
# Perform inverse DCT on each channel
```

```
# Merge Y, Cb, Cr channels
```

```
reconstructed_image = Image.merge('YCbCr', (reconstructed_y,  
reconstructed_cb, reconstructed_cr))
```

```
# Convert image back to RGB color space
```

```
final_image = reconstructed_image.convert('RGB')
```

```
# Save the compressed image
```

```
final_image.save('compressed_image.jpg')
```

```
# Display the compression ratio and reconstructed image
```

```
print("Compression Ratio:", compression_ratio)
```

```
final_image.show()
```

This is a simplified implementation, and it excludes some advanced features such as chroma subsampling and progressive encoding, which are commonly used in JPEG compression.

To explore wavelet-based compression, you can utilize the PyWavelets library, which provides various wavelet transforms and compression functionalities. You can use wavelet transforms like Discrete Wavelet Transform (DWT) or Stationary Wavelet Transform (SWT) to decompose the image into different frequency bands and then apply compression techniques like thresholding or bit-plane coding.

Implementing a full-fledged wavelet-based compression algorithm involves additional steps, including quantization, entropy coding, and the reverse process for decompression. You may need to delve deeper into the theory and algorithms behind wavelet-based compression to develop a complete implementation.

Remember to compare the performance of different compression algorithms based on factors such as compression ratio, image quality (using metrics like PSNR or SSIM), and computational complexity. Analyze the trade-offs between compression efficiency (higher compression ratio) and visual fidelity (lower image quality) to find the most suitable balance for your specific application.

Problem: 4

Here's an implementation in Python that demonstrates image restoration using Wiener filtering and total variation regularization:

```
import cv2
```

```
import numpy as np
```

```
# Load the degraded image
```

```
degraded_image = cv2.imread('degraded_image.jpg', 0) # Read image in grayscale
```

```
# Wiener Filtering
```

```
wiener_filtered = cv2.dft(degraded_image.astype(float),  
flags=cv2.DFT_COMPLEX_OUTPUT)
```

```
wiener_filtered = np.fft.fftshift(wiener_filtered)
```

```
# Specify the Wiener filter parameters
```

```
snr = 0.1 # Signal-to-Noise Ratio
```

```
k = 0.01 # Regularization parameter
```

```
H = np.conj(wiener_filtered) / (np.abs(wiener_filtered) ** 2 + snr)
```

```
wiener_restored = cv2.idft(np.fft.ifftshift(H * np.fft.fftshift(wiener_filtered))).real
```

```
# Total Variation Regularization
```

```
tv_restored = cv2.ximgproc.denoise_TVChambolle(degraded_image)
```

```
# Display the results
```

```
cv2.imshow('Degraded Image', degraded_image)
```

```
cv2.imshow('Wiener Filtered Image', wiener_restored.astype(np.uint8))
```

```
cv2.imshow('Total Variation Regularized Image', tv_restored.astype(np.uint8))
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Make sure to replace `'degraded_image.jpg'` with the actual filename and path of the degraded image you want to restore. This code loads the degraded image, performs Wiener filtering using the Discrete Fourier Transform (DFT), and applies total variation

regularization using the `denoise_TVChambolle()` function from OpenCV's extra modules (installable using `pip install opencv-contrib-python`).

You can adjust the parameters like the signal-to-noise ratio (`snr`) and regularization parameter (`k`) in the Wiener filter to optimize the restoration results based on your specific image and noise characteristics. Similarly, you can explore different denoising functions and parameters for total variation regularization to achieve the desired restoration outcome.

Evaluate the effectiveness of these restoration techniques by comparing the restored images with the degraded images visually and using metrics like Peak Signal-to-Noise Ratio (PSNR) or Structural Similarity Index (SSIM). You can also experiment with different types and levels of degradation, such as Gaussian noise, motion blur, or compression artifacts, to analyze the performance of the restoration methods under various scenarios.

Problem: 5

Here's a step-by-step implementation of building an image recognition system using feature extraction and classification with a convolutional neural network (CNN) in Python:

1. Dataset Preparation:

- Obtain a labeled dataset of images. You can use publicly available benchmark datasets like CIFAR-10, CIFAR-100, or ImageNet.
- Split the dataset into training and testing sets. The training set will be used for training the CNN model, while the testing set will be used for evaluation.

2. Preprocessing:

- Resize the images to a consistent size suitable for the CNN model.
- Normalize the pixel values to a specific range, such as `[0, 1]`, by dividing them by 255.
- Convert the labels into categorical format if they are not already.

3. Feature Extraction:

- Load a pre-trained CNN model (e.g., VGG, ResNet, Inception) from a deep learning framework like TensorFlow or Keras.
- Remove the classification layers from the CNN model, leaving the convolutional layers intact. These convolutional layers serve as feature extractors.
- Freeze the weights of the convolutional layers to prevent them from being updated during training.

4. Training:

- Create a new classification model by adding fully connected layers on top of the pre-trained CNN model.
- Compile the model with an appropriate loss function (e.g., categorical cross-entropy) and optimizer (e.g., Adam).
- Train the model using the training set. Iterate over the dataset for a specified number of epochs and update the model weights using backpropagation.
- Monitor the model's performance on the validation set and adjust the hyperparameters if necessary.

5. Evaluation:

- Evaluate the trained model on the testing set to measure its accuracy and performance.
- Calculate metrics such as accuracy, precision, recall, and F1-score to assess the model's performance.
- Optionally, visualize the classification results and confusion matrix to gain insights into the model's predictions.

6. Fine-tuning:

- If the initial model's performance is not satisfactory, you can try fine-tuning the model by unfreezing some of the earlier convolutional layers and continuing the training process with a lower learning rate.

7. Deployment:

- Save the trained model for future use or deployment in applications.
- Deploy the image recognition system, allowing it to accept new images and classify them using the trained model.

Remember to experiment with different CNN architectures, hyperparameters, and preprocessing techniques to optimize the accuracy and performance of your image recognition system. Additionally, consider using techniques like data augmentation (e.g., random cropping, flipping, rotation) to augment your training dataset and improve the model's generalization ability.

Code:

```
import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

from tensorflow.keras.datasets import cifar10


# Load and preprocess the CIFAR-10 dataset

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

num_classes = 10


# Normalize pixel values to [0, 1]

x_train = x_train.astype("float32") / 255.0

x_test = x_test.astype("float32") / 255.0


# Convert class labels to one-hot encoded vectors

y_train = keras.utils.to_categorical(y_train, num_classes)

y_test = keras.utils.to_categorical(y_test, num_classes)


# Define the CNN model

model = keras.Sequential(
```

```
[  
    layers.Conv2D(32, (3, 3), activation="relu", input_shape=(32, 32, 3)),  
    layers.MaxPooling2D(pool_size=(2, 2)),  
    layers.Conv2D(64, (3, 3), activation="relu"),  
    layers.MaxPooling2D(pool_size=(2, 2)),  
    layers.Flatten(),  
    layers.Dense(128, activation="relu"),  
    layers.Dense(num_classes, activation="softmax"),  
]  
)
```

Compile and train the model

```
model.compile(loss="categorical_crossentropy", optimizer="adam",  
metrics=["accuracy"])
```

```
model.fit(x_train, y_train, batch_size=64, epochs=10, validation_split=0.1)
```

Evaluate the model on the test set

```
test_loss, test_acc = model.evaluate(x_test, y_test)
```

```
print(f"Test Loss: {test_loss}")
```

```
print(f"Test Accuracy: {test_acc}")
```


